

CH 1 — Intro to Object Oriented Programming

Luke Lu • 2026-09-15

The course is taught in 3 perspectives

1. Programmer's perspective: efficiency, bug face
2. Designer's perspective: basic software engineering, design patterns
3. Compiler's perspective: what must a compiler to support a language feature



Info — C++ History

Invented in 1980s

C with classes is the earlier name of C++

Currently using C++20 with “modules” introduced

Hello World C++ Program

Program “01-intro/hello.cc”

```
import <iostream>;

using namespace std;

int main(){ // required for C++ to have return type as int for main()
    cout << "Hello World" << endl;
}
```

Command for running C++ programs:

Compile the system header

g++20h

g++15.2.0 #compiler using

-std=c++20 #standard

-fmodules-ts #use module tech specs

-c #separately compiling a part of a program

-x c++system-header

iostream #module name

Then compile the rest of the program

```
g++20

g++15.2.0      #compiler using

-std=c++20     #standard

-fmodules-ts   #use module tech specs

-wall         #warn all

-g            #debug

hello.cc

-o            # output
```

For this course, include, `stdio.h`, `printf` are banned.

Instead `std::cout << value <<std::endl` is used.

We can avoid `std::` by having `using namespace std`

“<<” is output operator, “>>” is input operator

Importing `iostream` gives access to three variables:

<code>std::cout</code>	type: ostream	<code>cout << x;</code> //print values stoerd in x on stdout
<code>std::cerr</code>	type: ostream	<code>cerr << "Error" << error << endl;</code> //print error message to stderr
<code>std::cin</code>	type: istream	<code>cin >> x;</code> //reads from stdin to store in x

Program “01-intro/plus.cc”

```
import <iostream>;

using namespace std;

int main(){
    int x, y;
    cin >> x >> y;
    cout << x+y << endl;
}
```



Info – I/O information

- Reading ints ignores leading white spaces
- After attempting to read, checking for success is important
- If a read fails, the expression “`cin.fail()`” will be true
 - If a read fails, all the subsequent reads fail. (unless specified)
 - When a read fails, we must acknowledge the failure and to attempt resuming reading using
 - `cin.clear();` to clear the flag
 - `cin.ignore();` to ignore the one that caused the flag to be raised.
- If a read fails due to end of input (eof), the expression “`cin.eof()`” will be true

Implicit/Automatic Conversion:

C++ will automatically treat `cin` as a boolean value: `!cin == cin.fail()`

Program “02-io/readInts.cc”

Goal: Read and print all ints from stdin. Stop in case of failure.

```
import <iostream>;
using namespace std;

int main(){
    int x;
    while(true){
        cin >> x; //expression statement, and evaluates to cin
        if(cin.fail()) break; //we can replace "cin.fail()" to "!(cin >> x)" or "!cin"
        cout << x << endl;
    }
}
```

Program “02-io/readInts5.cc”

Goal: Read and print all ints from stdin. Ignore all non-ints. Stop at eof.

```
import <iostream>;
using namespace std;

int main(){
    int x;
    while(true){
        if(cin >> x){
            cout << x << endl;
        } elif(cin.eof()){
            break;
        } else{
            cin.clear(); //clear the flag raised when reads fails for reading non-ints
            cin.ignore(); //ignore the one that caused the flag to be raised
        }
    }
}
```

I/O manipulators

```
int x = 95;  
cout << hex << x; //print in hexadecimal  
cout << dec; //go back to base-10  
  
//boolalpha, setprecision, skipws, noskipws, etc...
```